❼ . . . and if it doesn't exist, returns a 404 Not Found result and short-circuits the pipeline

❽ Executed after page handler execution (or short-circuiting)—not used in this example

The page filter is similar to the action filter equivalent. The most obvious difference is the need to implement three methods to satisfy the `IPageFilter` interface. You'll commonly want to implement the `OnPageHandlerExecuting` method, which runs after model binding and validation, and before the page handler executes.

A subtle difference between the action filter code and the page filter code is that the action filter accesses the model-bound action arguments using `context .ActionArguments`. The page filter uses `context.HandlerArguments` in the example, but there's also another option.

Remember from chapter 16 that Razor Pages often bind to public properties on the `PageModel` using the `[BindProperty]` attribute. You can access those properties directly instead of using magic strings by casting a `HandlerInstance` property to the correct `PageModel` type and accessing the property directly, as in this example:

```
var recipeId = ((ViewRecipePageModel)context.HandlerInstance).Id
```

This is similar to the way the `ControllerBase` class implements `IActionFilter` and `PageModel` implements `IPageFilter` and `IAsyncPageFilter`. If you want to create an action filter for a single Razor Page, you could save yourself the trouble of creating a separate page filter and override these methods directly in your Razor Page.

TIP I generally find it's not worth the hassle of using page filters unless you have a common requirement. The extra level of indirection that page filters